

Efficient Federated Learning of Mixed-Token Transformers for Cellular Feature Prediction

Daniel Becking*

Fraunhofer Heinrich Hertz Institute
Berlin, Germany
daniel.becking@hhi.fraunhofer.de

Jost Arndt*

Fraunhofer Heinrich Hertz Institute
Berlin, Germany
jost.arndt@hhi.fraunhofer.de

Ingo Frieze

Deutsche Telekom AG
Berlin, Germany

Karsten Müller

Fraunhofer Heinrich Hertz Institute
Berlin, Germany

Jackie Ma

Fraunhofer Heinrich Hertz Institute
Berlin, Germany

Thomas Buchholz

Deutsche Telekom AG
Berlin, Germany

Mandy Galkow-Schneider

Deutsche Telekom AG
Berlin, Germany

Wojciech Samek

Fraunhofer Heinrich Hertz Institute
Technical University Berlin
BIFOLD
Berlin, Germany

Detlev Marpe

Fraunhofer Heinrich Hertz Institute
Berlin, Germany

Abstract—In telecommunications, Autonomous Networks (ANs) automatically adjust configurations based on specific requirements (e.g., bandwidth) and available resources. These networks rely on continuous monitoring and intelligent mechanisms for self-optimization, self-repair, and self-protection, nowadays enhanced by Neural Networks (NNs) to enable predictive modeling and pattern recognition.

In this work, we propose a neural architecture called *Mixed-Token Transformer* (MTT) that can process and predict both numerical and textual tokens representing various mobile network features (e.g., ping, SNR or band frequency). We train the models using a Federated Learning (FL) approach, enabling multiple AN cells — each equipped with an MTT — to collaboratively learn from cellular data while preserving data privacy.

Since FL involves frequent transmission of neural network updates, it requires an efficient, standardized compression strategy for reliable communication. To address this, we investigate *NNCodec*, our implementation of the ISO/IEC Neural Network Coding (NNC) standard. Our experimental results on the *Berlin V2X* dataset demonstrate that *NNCodec* achieves transparent compression (i.e., negligible performance loss) while reducing communication overhead to below 1%, showing the effectiveness of combining NNC with FL in collaboratively learned autonomous mobile networks. Our proposed MTT architecture speeds up model inference by $\approx 5\times$. Code is available at: github.com/JostArndt/NNCodec-FL-MixedTokenTransformer.

Index Terms—Efficient Transformer Models, Autonomous Mobile Networks, Federated Learning, Neural Network Coding, Cellular Data, QoS Prediction

I. INTRODUCTION

With the advancement of 5G and beyond, assessing Quality of Service (QoS) and Quality of Experience (QoE) has become essential. Wireless communications are expected to integrate advanced machine learning technologies, and one promising strategy is to equip mobile network cells with

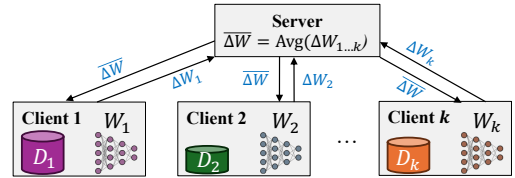


Fig. 1. Federated averaging differential weight updates ΔW_i .

artificial Neural Networks (NNs) that collaboratively learn in a Federated Learning (FL) setup, enhancing overall QoS and QoE. In FL, a shared NN is trained across multiple client devices, each utilizing its own data subset (see Fig. 1). A central server coordinates the training process by aggregating updates from clients and redistributing them. While FL offers data protection by keeping data local, it faces challenges like communication overhead due to frequent NN data exchanges, often constrained by bandwidth. To address this, effective data compression is crucial to minimize communication overhead and latency. Two main strategies are: (i) reducing the frequency of weight updates (communication delay) by performing multiple local training iterations before transmitting updates, and (ii) compressing transmitted data through parameter reduction (e.g., pruning, sparsification, or decomposition), precision reduction (quantization), and lossless compression (e.g., entropy coding or sparse matrix formats).

In this work, we employ FL to train Transformer-based language models on cellular data from the *Berlin V2X* dataset [1] to predict quality features such as ping, RSSI, and jitter, based on input features like MCS (Modulation and Coding Scheme), Tx_Power, frequency, GPS features and environmental side information. For this task,

*These authors contributed equally to this work.

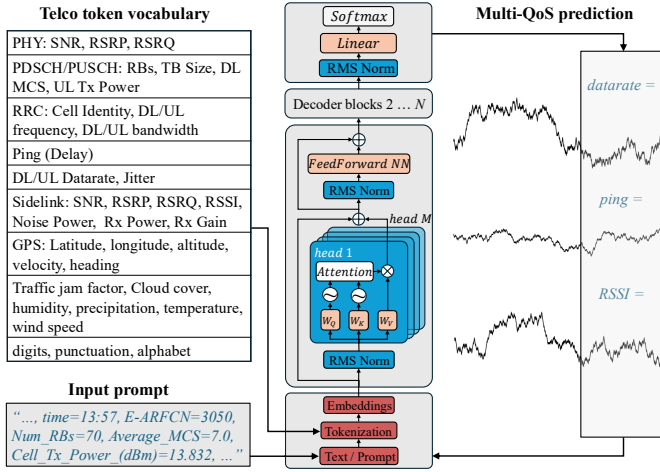


Fig. 2. Scheme of Transformer-based cellular feature predictions.

we developed a specialized data pre-processing method to tokenize tabular data for use with Transformer architectures. We compare a baseline Transformer that predicts numerical data digit-by-digit with a proposed, more efficient model that incorporates a dedicated embedding layer and different output heads for directly processing and predicting numerical values. For an efficient communication, the neural data exchanged between clients and server in the federated learning setup is compressed using technology from the ISO/IEC Neural Network Coding (NNC) standard [2].

A. Our Contributions

We hereby want to briefly list our contributions

- A new use case for intelligent mobile networks leveraging the flexibility of language models to process diverse input modalities and predict or complement multiple cellular features or configurations jointly.
- Mixed tokenization process for multi-modal data, significantly reducing the number of tokens per sample.
- Customized Mixed-Token Transformer (MTT) architecture, mixing regression and classification to achieve approximately $5\times$ faster runtime (shorter context lengths) and improved predictive performance.
- Integration of multiple virtual AN cells into an FL setup, enabling collaborative, privacy-preserving training across geographic areas, supporting modularity / transferability.
- Efficient FL communication through compressing incremental neural updates with *NNCodec* [3], our implementation of the NNC standard.

II. METHOD

We begin by introducing the Berlin V2X [1] cellular dataset, which we deploy to simulate a potential use case in autonomous telecommunication networks. The “Where Are the Cellular Data?” survey [4] convincingly demonstrates that publicly available cellular datasets with meaningful features are extremely scarce. The Berlin V2X dataset is a notable exception, containing a wide range of correlated features that

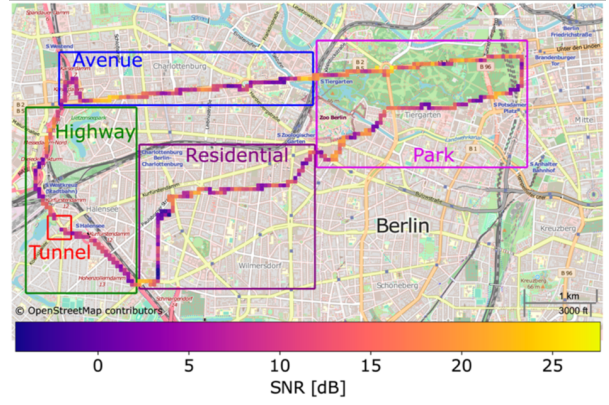


Fig. 3. Berlin V2X measurement route including five urban environments.

also [5] found crucial for accurate bandwidth predictions when compared to other public cellular datasets. From the dataset, we derive a custom tokenizer and a tokenization process, described in the following section. Next, we present the Transformer-based language models, that we utilize to learn correlations in cellular data and to predict missing features of a given input sequence that may contain a concatenation of the Berlin V2X [1] dataset’s features, as illustrated in Fig. 2. We then describe the architectural extensions made to the Transformer to enable distinct processing of textual and numerical data. Finally, we detail the compression pipeline that supports communication-efficient federated learning across multiple intelligent network cells.

A. Berlin V2X Data and Pre-processing

The Berlin V2X dataset [1] provides high-resolution, GPS-located wireless measurements in various urban settings along a 17.2 km long route through Berlin city, involving up to four cars driving the route 17 times over three days. The dataset contains 159 features including information on physical layer parameters, cellular radio resource management, QoS parameters, GPS positioning data, and additional context like traffic and weather conditions.

For our analysis, we filter data points connected to operator 1 (i.e., Deutsche Telekom’s network), yielding 103,554 GPS-located samples. A random 10% is reserved for testing. The remaining data is split according to the associated area segments ‘Residential’, ‘Park’, ‘Avenue’, ‘Highway’, ‘Tunnel’, as highlighted in Fig. 3, which results in 28,017, 33,487, 19,747, 10,390, and 475 training samples for the five respective AN clients participating in FL.

For language model training, input data is typically divided into discrete units called tokens. To address the unique characteristics of the Berlin V2X dataset, we developed a specialized token vocabulary designed to accurately capture its cellular communication patterns. Each dataset sample contains mixed data types; for example, the sequence “PCell_RSRQ_max = -4.775” (a string) combines textual tokens (“PCell_RSRQ_max”, “=”) and a numerical token (-4.775). Our proposed tokenization strategies to handle

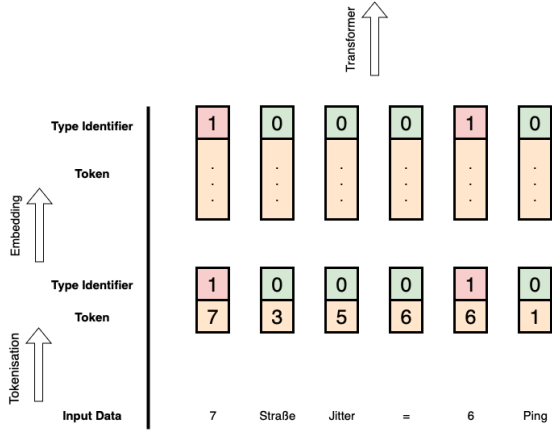


Fig. 4. Tokenization of the mixed textual- and regression-input tokens.

such data will be detailed in the following Subsection. It is important to note, that the values of some features (such as street names) are also textual. In addition, each sample contains a different number of entries, as sensor outputs vary depending on the specific sensor and sample, resulting in frequent absence of data for certain measurements. During pre-processing, we applied feature-wise standardization of the numerical values. In addition, we excluded samples with parameters exceeding six standard deviations from the mean. These outliers represent approximately 3% of the dataset.

B. Tokenization Strategies

We seek to compare two distinct approaches to data processing, beginning with different tokenization strategies.

1) *Tokenization for Digit-Wise Transformers:* As a naive approach, we tokenize every input sequence using the *SentencePiece* [6] tokenizer, which we specifically trained on the vocabulary of the Berlin V2X dataset. For numerical values, we employ a digit-by-digit tokenization strategy, e.g., -7.445 is split into $-$, 7 , $.$, 4 , 4 , 5 (series of strings), and subsequently processed through the tokenizer.

2) *Tokenization for Mixed-Token Transformers:* We apply different tokenizations for both token types, i.e., textual and numerical. Furthermore, we equip every token with information about the original modality of the token itself. To do so, we create two-dimensional tokens: The token type information is encoded in the first element of every token through a binary representation. Textual tokens are assigned a 0 and numbers are assigned a 1. Appending information on the type of the token to the token itself has been done before, e.g., for graphs [7], or tabular data [8]. Additionally, we append the actual token, originating from the two different tokenizers. As a tokenizer for textual tokens we use the *SentencePiece* tokenizer from the approach above. As a numerical tokenizer, we use the pre-processed (i.e., normalized) values directly. A schematic drawing of this tokenization can be found in Fig. 4. Since this tokenization scheme does not decompose the numerical parameters into multiple tokens, it results in shorter sequences per sample, allowing us to reduce

the maximum sequence length in the following modeling approaches. This is especially important since computational complexity of Transformer models scales quadratically with sequence length.

C. Efficient Transformer Models for Cellular Data

Transformer-based models demonstrate remarkable performance in capturing complex correlations within data through the attention mechanism not only on language data [9], but also many other modalities, such as images [10], weather [11], graphs [7], and tabular data [12]. They effectively identify relationships between tokens in the input sequence, which enables the model to track context. This comprehension allows the model to generate relevant responses by predicting the probability distribution across its entire vocabulary (i.e., set of unique tokens the model can handle) and determining the most probable next token. While our application focuses on tabular data, we believe it is the first to simultaneously handle mixed numeric and categorical input features while performing both classification and regression tasks in a single forward pass, marking a key difference to tabular architectures known to the authors.

Due to the nature of our task — combining tokens of mixed modalities with varying in- and output-lengths in both regression and classification settings — we consider Transformer-based architectures to be the overwhelmingly most promising approach. The extensive research conducted in recent years has demonstrated their outstanding performance across diverse data types and tasks, along with many optimized implementations for different use cases.

Our work studies the technical feasibility of such models to capture correlations and dependencies in cellular data by converting it from tabular format into strings and subsequently into tokens. Then, each input sequence represents a concatenated set of cellular features as collected in the Berlin V2X dataset. We utilized a lightweight version of Llama 2 [13], consisting only of Transformer decoder layers, built on the

TABLE I
TINY LLAMA CONFIGURATIONS AND RESULTING MODEL SIZES.

Llama configuration	Size = 1	Size = 2
dim^a	288	512
n_layers^b	10	10
n_heads^c	6	8
$n_kv_heads^d$	6	8
$vocab_size^e$	237	237
$max_seq_len^f$	[528, 832]	[528, 832]
Number of tensors	107	107
Tensor elements	10,429,446	32,926,758
Memory	41.72 MB	131.71 MB

- a Size of embedding vectors (base size of the hidden representation).
- b Number of transformer blocks, each comprising multi-head attention and Feed-forward neural network (FFN) components.
- c Number of key-value heads (which can be decoupled from the number of query heads).
- d Number of attention heads in the multi-head self-attention mechanism.
- e Size of the vocabulary, representing the number of unique tokens the model can handle.
- f Maximum input sequence length the model can process.

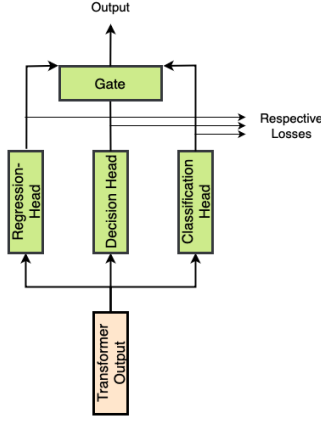


Fig. 5. Routing of the output and losses through three different heads. During training, the losses are computed prior to the actual gating mechanism.

llama.c [14] implementation. For both tokenization-strategies we used different embedding and output layers and heads; while the rest of the model stays the same for a better comparison. We used two different sizes of Llama, with parameters configurations explained in Table I.

1) *Digit-Wise Transformer*: As a naive approach, we just follow the standard architecture of Llama without any adaptations. We train the model using the cross-entropy loss for all tokens.

2) *Mixed-Token Transformer*: For the mixed-modality approach, we employ two distinct embedding layers on the respective tokens: for the textual embeddings, we use a learnable embedding layer to convert the tokens to vectors of size $dim - 1$. This layer and the pre-softmax linear output layer share weights, a technique called weight tying [15], following the original Transformer implementation [9]. The numerical tokens are processed through a separate linear layer, projecting scalar values to dimension $dim - 1$. Following the two parallel embedding layers, we merge the sequences of embedded tokens in the previous order, to receive a single sequence again. On this sequence we use a single standard positional encoding followed by the Llama implementation from above. After the last Transformer layer, we pass the output through three parallel output heads: a textual output head, a numerical output head and a decision output head. The decision output head supplies binary outputs that can be used in a gating mechanism to decide if the output token shall be passed through either the textual- or numerical output-head. The textual output head is just the plain output-layer projecting from the size $dim - 1$ to $vocab_size$ of the original Llama implementation, to produce the logits. The numerical output head is a multilayer perceptron (MLP) consisting of three layers with hidden dimension of dim and output dimension 1 to produce numerical (scalar) values. The decision head has the same format, followed by a sigmoid and a threshold function, to produce a binary output. The mechanism can be seen in Fig. 5.

During training, we calculate three losses independently:

$\mathcal{L}_{text}$ is the loss for textual tokens, for which we use the cross-entropy loss, for all text tokens per batch. $\mathcal{L}_{numeric}$ is the mean squared error loss between all numerical tokens in each batch. As decision loss $\mathcal{L}_{decision}$, we employ the binary cross-entropy loss between the decision head and the actual data type (text or number) of each token in the dataset, effectively combining a regression and a classification problem. Due to the varying difficulty of the tasks and the different scale of the losses, the losses can not be simply summed up for training. However, using Gradient Normalization [16] allows us to efficiently train all heads and the underlying Transformer at the same time by adaptively balancing the losses through their gradient. During inference, the model predicts multiple tokens in an autoregressive forward loop without computing these losses; only the gated output token is used and transformed to distinguish between textual and numeric tokens.

D. Efficient Federated Learning for Intelligent Network Cells

Efficient FL requires a suitable combination of model complexity, data compression, and local learning. We integrated *NNCodec* [3], our open-source implementation of the NNC standard (ISO/IEC 15938-17), to compress communication among five Transformers participating in an FL task to predict cellular network features. For encoding the differential updates ΔW_i between $i \in \{1, \dots, 5\}$ clients and the server, we use NNC's encoder-decoder architecture as detailed in [17]. The process begins with parameter reduction techniques such as sparsification and pruning. These methods offer significant compression gains, especially for differential updates, due to their more centralized distribution, i.e., with higher probability of smaller absolute values. Quantization follows, controlled by a quantization parameter (qp), and concludes with optimized arithmetic coding using DeepCABAC. On the decoder side, the process is reversed, beginning with entropy decoding, followed by parameter reconstruction and rescaling. The NNC standard also provides high-level syntax for coordinating decoded incremental weight updates in distributed learning: The Parameter Update Tree (PUT) [17] offers a referencing mechanism to uniquely identify coded entities relative to previous versions, supporting flexible communication, partial NN updates, (a)synchronous settings, client failures, and new client integration — making it well-suited to the requirements of the use case presented above.

1) *Parameter reduction*: The sparsification and pruning pipeline involves tensor-wise statistics-adaptive sparsification (unstructured), neuron-wise sparsification (structured), and pruning, which removes sparsified structures like rows, columns or updates of entire layers. $\Delta W_\rho^{(t)} = W_\rho^{(t)} - W_\rho^{(t-1)}$ represents the differential model update after multiple local training iterations. ρ indexes the different parameters of the Transformer architecture, e.g., the attention head's QKV matrices, and t denotes the current communication round. In a first step, all absolute weight update values $|\Delta W_\rho^{(t)}|$ below the threshold θ_{qp} are set to zero, where

$$\theta_{qp} = \frac{4 + (qp \bmod 4)}{2} \cdot 2^{\lfloor \frac{qp}{4} \rfloor - 2}. \quad (1)$$

θ_{qp} ensures that sparsification zeros out values beyond those that will be assigned to zero by the subsequent quantization process. Next, the weight updates $\Delta W_\rho^{(t)}$ are further sparsified based on a threshold θ_ρ , which is determined by the mean values μ and standard deviations σ of the model’s parameter-wise weight updates: $\forall_\rho \theta_\rho = \max(|\mu_\rho - \delta \cdot \sigma_\rho|, |\mu_\rho + \delta \cdot \sigma_\rho|)$, s.t. $\theta_\rho > \theta_{qp}$. Here, δ is configured to achieve a specific target sparsity like 60% in $\Delta W_\rho^{(t)}$. Structured sparsification focuses on zeroing adjacent groups of weight update elements. In Transformer architectures, this translates to setting entire rows or columns of the query (ΔW_Q), key (ΔW_K), or value (ΔW_V) matrix updates to zero. Updates to output neurons in the FFN’s linear layers can also be set to zero. The criterion for sparsifying a neuron’s update is based on its absolute arithmetic mean value within the tensorial update. After sorting the mean values of all rows of a tensor update in ascending order, the top- ϵ rows are set to zero. We select ϵ so that all rows with mean values below 90% of the average of all row mean values are sparsified. One-dimensional parameter types, such as bias parameters (and their updates), are excluded from sparsification.

2) *Quantization*: Each weight update element $\Delta W_{\rho,i}^{(t)} \in \Delta W_\rho^{(t)}$ is uniformly quantized to the nearest multiple of $2 \cdot \theta_{qp}$, as per equation (1), by $\hat{\Delta W}_{\rho,i}^{(t)} = \text{round}\left(\frac{\Delta W_{\rho,i}^{(t)}}{2 \cdot \theta_{qp}}\right) \cdot 2 \cdot \theta_{qp}$. Smaller values of qp lead to finer quantization, typically resulting in larger bitstream sizes and reduced performance degradation. Note that the uniform quantization used in NNC is not restricted to a specific number of quantization levels; it is only limited by the maximum representable value of an int32. The decoder’s dequantization step reconstructs the original floating-point approximation by $\forall_{\rho,i} \hat{\Delta W}_{\rho,i}^{(t)} \cdot 2 \cdot \theta_{qp}$.

3) *Entropy Coding*: As a result of the sparsification and quantization techniques described above, many rows in the Transformer’s ΔW_Q , ΔW_K , and ΔW_V matrices are set to zero. This sparsity can be exploited during entropy coding — e.g., by omitting these zero rows, a technique referred to as Row Skipping in the NNC standard.

At the core of the NNC compression pipeline is DeepCABAC (Context-Adaptive Binary Arithmetic Coding), which operates in three stages: 1) binarization, where each non-binary element to be encoded (e.g., a quantized weight) is converted into a sequence of binary symbols, known as bins; 2) context modeling, where a probability model is assigned to each bin based on its unique binarization context; and 3) binary arithmetic coding, which compresses the bins according to their probability estimates. Each context model uses a backward-adaptive probability estimator that maintains an internal state. After processing a bin, the state is updated based on the processed value. Such adaptive modeling allows DeepCABAC to efficiently adjust to varying input distributions. We additionally incorporated Temporal Context Adaptation [17] to exploit temporal dependencies across successive incremental updates during the context modeling stage, thereby improving compression performance

and enabling efficient neural communication in FL.

III. EXPERIMENTS

In this section, we first describe our experimental setup. We then present results on compression strategies aiming to improve communication efficiency in FL, followed by a comparison of different tokenization methods and Transformer architectures for efficient local cellular feature prediction. Given that LSTMs dominated sequential modeling prior to the emergence of Transformer architectures, we additionally ran an exploratory experiment substituting the Transformer backbones with LSTMs [18] of comparable size (same embedding/hidden dim and $1.5 \times n_layers$ depth).

A. Experimental Setup

The experimental setup involved varying hyperparameters as follows: model size values of 1, 2 according to Table I, initial learning rates (lr) of $3e-4$, $5e-5$, different number of batches used for local training per communication round ($max_batches \in [150, 300]$), quantization parameter variations from $qp \in [-30, -26, -22, -18]$ and sparsity rates $\in [0.0, 0.6, 0.75, 0.8]$. An Adam optimizer with cosine annealing learning rate and a $batch_size$ of 8 was used for all experimental runs. As a federated learning software framework we utilized *Flower* [19] and compressed all communication, up- and downstream, using *NNCodec*. All experiments ran for $t \in [0, \dots, 30]$ FL communication rounds. We employed a weighted federated averaging (FedAvg) strategy at the server for model aggregation. This approach accounts for the number of samples per client, which is essential given the heterogeneous data distribution (see Section II-A). All clients were sampled in random order across all communication rounds. Additionally, we used error accumulation (“residuals” R) in all experiments, i.e., the weight update error, which arises from parameter reduction and quantization, is computed as the difference between the uncompressed weight update and its compressed counterpart $R_\rho^{(t)} = \hat{\Delta W}_\rho^{(t)} - \Delta W_\rho^{(t)}$ and added to the weight update of the subsequent communication round prior to compression $\Delta W_\rho^{(t+1)} = \Delta W_\rho^{(t+1)} + R_\rho^{(t)}$. This ensures that even small weight changes in certain areas of the network can accumulate over multiple training rounds and be sent once they are above a certain threshold.

B. Compression Results

In the uncompressed baseline setting, the digit-wise Transformers achieved over 89% top-1 accuracy for next-token prediction and perplexities below 1.35, demonstrating their

TABLE II
SELECTED TRANSPARENT CODING RESULTS FOR DIFFERENT MODEL SIZES.

Size	lr	qp	Sparsity	Performance		Data [MB]		Comp. Ratio [%]	Perf. Degradation	
				Acc [%]	PPL	Uncomp.	Comp.		Acc [%]	PPL
1	5e-5	-22	0%	89.70	1.35	10,095.84	25.10	0.25	-0.33	+0.00
1	3e-4	-18	80%	89.88	1.34	10,095.84	67.16	0.67	+0.03	-0.00
2	5e-5	-22	60%	89.73	1.35	31,874.56	49.95	0.16	-0.34	+0.00
2	3e-4	-22	0%	89.70	1.35	31,874.56	487.12	1.52	+0.11	-0.00

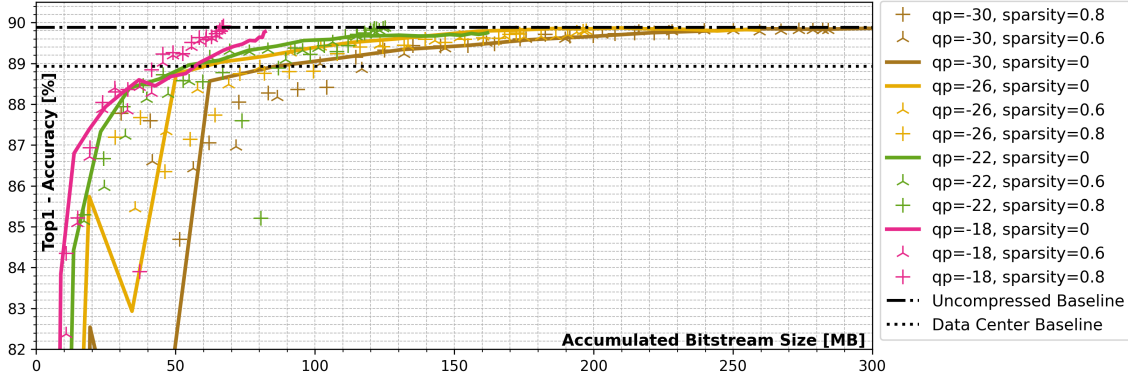


Fig. 6. Coding performance of various qp and sparsity values with a model size of 1, learning rate $3e-4$, y-axis limited to values $\geq 82\%$.

TABLE III
ENCODER/DECODER COMPLEXITIES.

Size	qp	Sparsity	Enc Time ^a	Dec Time ^b	Enc+Dec Time/ t ^c	Time/ t ^d
1	-22	0%	1.63	0.27	12.48	110.63
1	-22	75%	0.95	0.14	7.04	99.17
2	-22	0%	4.60	0.82	35.84	225.93
2	-22	75%	2.82	0.40	20.91	203.97

^aaverage time in seconds to encode one differential model update ΔW .

^baverage time in seconds to decode one differential model update ΔW .

^caverage time in seconds for overall encoding and decoding per communication round t .

^daverage time in seconds per round t , including training, coding, and aggregation at server.

TABLE IV
COMPARISON OF MODELS.

Model	max_seq_len	MAPE ^a	# inference ^b	time ^c
Digit-wise, size 1	832	26.96	268	7.25
Digit-wise, size 2	832	27.36	268	9.96
Mixed-Token, size 1	528	25.47	82	1.68
Mixed-Token, size 2	528	23.46	82	1.87

^aMAPE excluding *jitter* due to outliers, see Fig. 7.

^baverage number of forward passes to complete a sample sequence with QoS predictions, considering three decimal tokens per numeric feature in the digit-wise case.

^caverage number of seconds for test-set sample completion on a MacBook Pro M3.

ability to capture correlations in cellular data and predict missing features. Various compressed settings are shown in Fig. 6. Table II details selected runs and compression efficiency. Here, for model sizes 1 and 2, one selected case achieved transparency with minimal communicated data, while another surpassed baselines at the cost of additional data. Here, “transparency” is defined as a maximum of 1% relative performance degradation compared to the uncompressed scenario (e.g., -0.89% accuracy and $+0.01$ perplexity). In conclusion, compression rates below 1% of the original size with negligible performance degradation are feasible in the presented use case, saving dozens of gigabytes of data transfer.

Table III compares the computational complexity of encoding and decoding operations relative to the total training time per communication round t . Encoding and decoding times were measured on a single CPU, whereas training time is measured on a GPU and executed in parallel. On average, server- and client-side coding processes together account for 7–15% of the total communication round runtime, depending on model size and compression scheme used.

C. Comparison Between Transformer Approaches

We present the performances of our models as mean absolute percentage errors (MAPE) on different QoS features. As shown in Fig. 7, the LSTMs underperform the Transformers at matched compute and parameter budgets, demonstrating their limitations and confirming the superior efficiency and versatility of Transformers for our specific task. Consequently, our remaining analysis focuses exclusively on the two Transformer variants. Both models individually display strengths and weaknesses across different features. While MTT shows

significant deficiency and instability for outliers in the QoS feature *jitter*, both models generally perform similarly well and maintain stability on the other QoS features.

Fig. 7 demonstrates that autoregressive predictions suffer from error propagation, where prediction errors accumulate for features appearing later in the sequence. To validate this hypothesis, we evaluated the MAPE on only the datarate feature using two different input contexts: first with the standard input context feature set which includes all but the QoS features depicted in Fig. 7, and second with extended context including QoS features up until the datarate feature itself. The extended context approach results in significant error reduction across all four models, reducing MAPE by up to $\approx 75\%$ (e.g., from 83% to 21% for the digit-wise Transformer of size 2). This yields a mean absolute error (MAE) of 8.5 Mbps in datarate, which is comparable to the performance of state-of-the-art single-feature prediction models that, as opposed to our approach, incorporate feature history as model input during inference [5]. The distribution of relative losses (MAPE) for a single exemplary parameter is shown in Fig. 8, indicating that the losses are normally distributed for both models.

We see that the performance of MTT improves with increasing model size across most QoS features, and the larger MTT exhibits a lower MAPE compared to the digit-wise model across several parameters. Furthermore, MTT performs up to 5 times faster in terms of runtime in comparison to the naive approach, as shown in Table IV. This is due to the decreased number of necessary inference steps per sample, and shorter context lengths. These results indicate that in-

dividual tokenization-strategies and customized architectures can drastically increase the performance on real-world cellular data applications.

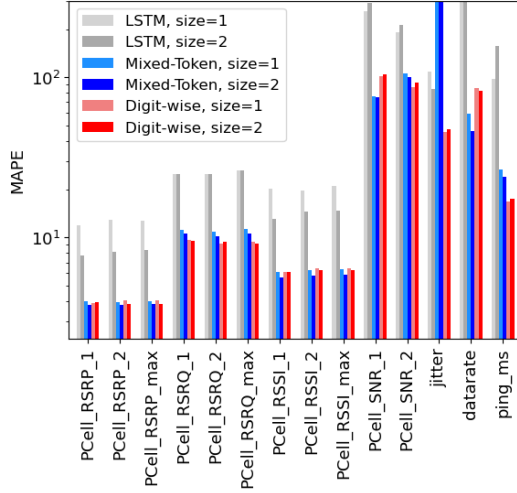


Fig. 7. Comparison of the models of both size 1 and size 2 for different numerical QoS features.

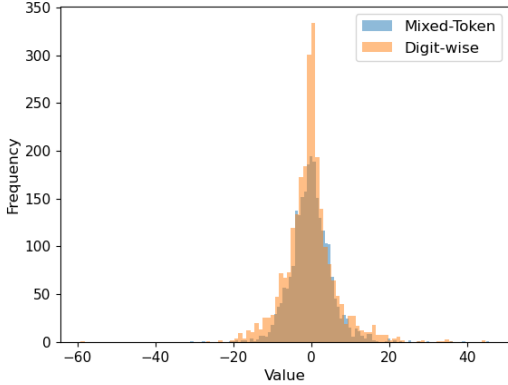


Fig. 8. Comparison of the MAPE of the MTT and digit-wise Transformer, both of size 2.

IV. CONCLUSIONS AND FUTURE WORK

In this paper, we presented an efficient FL scenario for virtual autonomous network cells, using the Berlin V2X cellular dataset. By employing compact Transformer models for QoS prediction, we explored a novel use case that can hopefully enhance future autonomous mobile networks.

Our approach proposed a Mixed-Token Transformer architecture and custom tokenization strategy that significantly reduced computational complexity and runtime while improving model performance. Furthermore, we integrated *NNCodec* to compress neural update transmissions, achieving compression ratios below 1% with negligible performance degradation — enabling efficient, privacy-preserving communication across AN clients. These findings demonstrate the feasibility of deploying *NNCoded* language models in FL settings using

real-world cellular data and pave the way for further research in autonomous network intelligence.

Future work includes reversing the current processing pipeline to predict optimal configuration parameters (e.g., modulation and coding schemes, bandwidths, or frequencies) based on environmental context and target QoS requirements. This would enable dynamic adaptation of network behavior based on user needs. Another promising direction involves leveraging historical per-feature data, especially when larger cellular datasets will become publicly available in the future. Furthermore, exploring domain adaptation and personalization techniques could enhance the ability of client cells to accommodate local heterogeneity and specialized deployment scenarios, including challenging environments such as tunnel environments. By pursuing these directions, we aim to advance the performance, adaptability, and usability of our efficient federated learning system for cellular networks.

REFERENCES

- [1] R. Hernangómez *et al.*, “Berlin V2X: A Machine Learning Dataset from Multiple Vehicles and Radio Access Technologies,” in *IEEE VTC2023-Spring*, Florence, Italy, Jun. 2023, pp. 1–5.
- [2] ISO, “Information technology — Multimedia content description interface — Part 17: Compression of neural networks for multimedia content description and analysis,” International Organization for Standardization, Geneva, CH, Standard ISO/IEC 15938-17:2024, 2024.
- [3] D. Becking *et al.*, “NNCodec: An Open Source Software Implementation of the Neural Network Coding ISO/IEC Standard,” in *ICML Neural Compression Workshop: Information Theory to Applications*, Jul. 2023.
- [4] M. Amini, R. Stanica, and C. Rosenberg, “Where are the (cellular) data?” *ACM Computing Surveys*, vol. 56, no. 2, Sep. 2023.
- [5] B. Denizer and O. Landsiedel, “Bandseer: Bandwidth prediction for cellular networks,” in *2024 IEEE 49th Conference on Local Computer Networks (LCN)*, 2024, pp. 1–8.
- [6] T. Kudo and J. Richardson, “Sentencepiece: A simple and language independent subword tokenizer and detokenizer for neural text processing,” *CoRR*, vol. abs/1808.06226, 2018.
- [7] J. Kim *et al.*, “Pure transformers are powerful graph learners,” in *NeurIPS*, 2022.
- [8] Y. Gorishniy *et al.*, “Revisiting deep learning models for tabular data,” in *NeurIPS*, 2021.
- [9] A. Vaswani *et al.*, “Attention is all you need,” in *NeurIPS*, 2017.
- [10] A. Dosovitskiy *et al.*, “An image is worth 16x16 words: Transformers for image recognition at scale,” in *ICLR*, 2021.
- [11] K. Bi *et al.*, “Accurate medium-range global weather forecasting with 3D neural networks,” *Nature*, vol. 619, no. 7970, pp. 533–538, Jul. 2023.
- [12] S. Somvanshi *et al.*, “A survey on deep tabular learning,” *arXiv preprint arXiv:2410.12034*, 2024.
- [13] H. Touvron *et al.*, “Llama 2: Open Foundation and Fine-Tuned Chat Models,” Jul. 2023, arXiv:2307.09288.
- [14] A. Karpathy, “llama2.c,” <https://github.com/karpathy/llama2.c>, 2023, accessed: April 24, 2025; License: MIT.
- [15] O. Press and L. Wolf, “Using the output embedding to improve language models,” in *Proceedings of the 15th Conference of the European Chapter of the Association for Computational Linguistics: Volume 2, Short Papers*, 2017, pp. 157–163.
- [16] Z. Chen *et al.*, “GradNorm: Gradient normalization for adaptive loss balancing in deep multitask networks,” in *ICML*, 2018.
- [17] D. Becking *et al.*, “Neural Network Coding of Difference Updates for Efficient Distributed Learning Communication,” in *IEEE Transactions on Multimedia*, vol. 26, 2024, pp. 6848–6863.
- [18] S. Hochreiter and J. Schmidhuber, “Long short-term memory,” *Neural computation*, vol. 9, no. 8, pp. 1735–1780, 1997.
- [19] D. J. Beutel *et al.*, “Flower: A Friendly Federated Learning Research Framework,” Mar. 2022, arXiv:2007.14390.